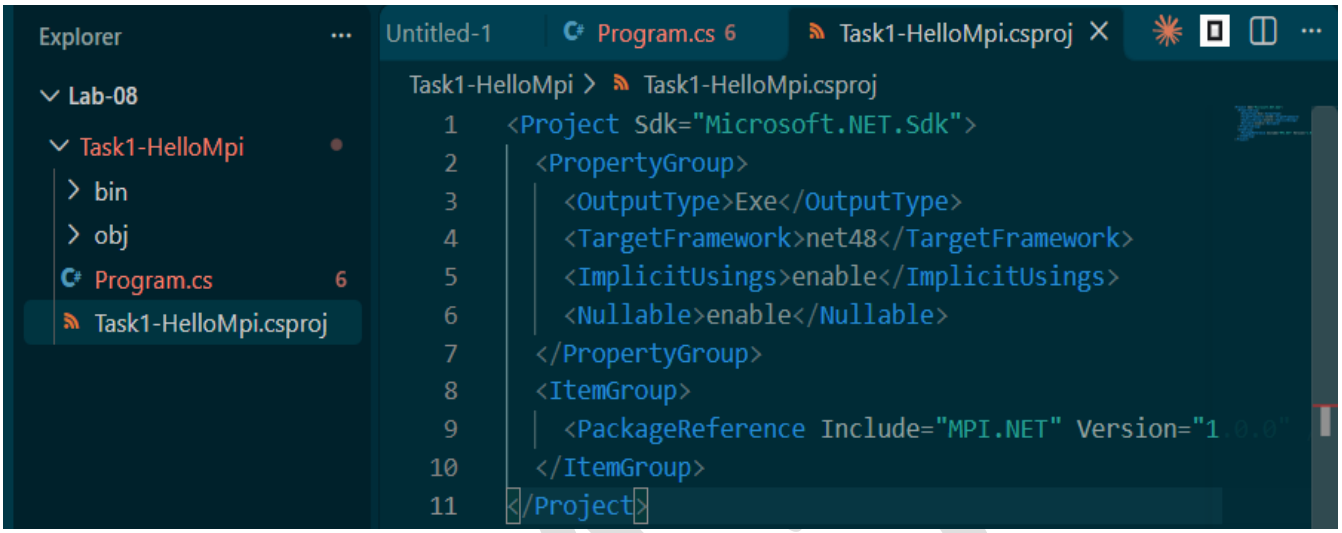


Name:	Muhammad Hasnat Fakhar
Class/Section:	BSCSev-F23-C
Reg ID:	232436
Subject:	Parallel and Distributing Computing
Task:	LAB TASK 07

LAB TASK 01

Code

1.



2.

```
using System;
using MPI;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        using (new MPI.Environment(ref args))
        {
            Communicator world = Communicator.world;
            int rank = world.Rank;
            int size = world.Size;

            // Yahan fix kiya gaya hai: System.Environment likha hai
            string hostname = System.Environment.MachineName;

            // 1. Unordered output (Direct print)
            Console.WriteLine($"[Unordered] Rank {rank} of {size} on {hostname}");

            // 2. Ordered output through Message Passing
            if (rank != 0)
            {
                string msg = $"Rank {rank} of {size} on {hostname}";
                world.Send(msg, 0, 99);
            }
            else
            {
                Console.WriteLine("\n--- Ordered Output ---");
                Console.WriteLine($"Rank 0 of {size} on {hostname}");

                for (int i = 1; i < size; i++)
                {
                    string msg = world.Receive<string>(i, 99);
                    Console.WriteLine(msg);
```

Output

```
PS E:\Lab-08> cd E:\Lab-08\Task1-HelloMpi
>> dotnet build -c Release
>> mpiexec -n 4 .\bin\Release\net48\Task1-HelloMpi.exe
>> mpiexec -n 6 .\bin\Release\net48\Task1-HelloMpi.exe
>> mpiexec -n 8 .\bin\Release\net48\Task1-HelloMpi.exe
Restore complete (0.4s)
  Task1-HelloMpi net48 win-x86 succeeded (0.4s) → bin\Release\net48\Task1-HelloMpi.exe

Build succeeded in 1.6s
[Unordered] Rank 0 of 4 on DESKTOP-AELKMIU

--- Ordered Output ---
Rank 0 of 4 on DESKTOP-AELKMIU
[Unordered] Rank 3 of 4 on DESKTOP-AELKMIU
[Unordered] Rank 2 of 4 on DESKTOP-AELKMIU
[Unordered] Rank 1 of 4 on DESKTOP-AELKMIU
Rank 1 of 4 on DESKTOP-AELKMIU
Rank 2 of 4 on DESKTOP-AELKMIU
Rank 3 of 4 on DESKTOP-AELKMIU
[Unordered] Rank 4 of 6 on DESKTOP-AELKMIU
[Unordered] Rank 2 of 6 on DESKTOP-AELKMIU
[Unordered] Rank 5 of 6 on DESKTOP-AELKMIU
[Unordered] Rank 0 of 6 on DESKTOP-AELKMIU

--- Ordered Output ---
Rank 0 of 6 on DESKTOP-AELKMIU
[Unordered] Rank 1 of 6 on DESKTOP-AELKMIU
[Unordered] Rank 3 of 6 on DESKTOP-AELKMIU
Rank 1 of 6 on DESKTOP-AELKMIU
Rank 2 of 6 on DESKTOP-AELKMIU
Rank 3 of 6 on DESKTOP-AELKMIU
Rank 4 of 6 on DESKTOP-AELKMIU
Rank 5 of 6 on DESKTOP-AELKMIU
[Unordered] Rank 4 of 8 on DESKTOP-AELKMIU
[Unordered] Rank 1 of 8 on DESKTOP-AELKMIU
[Unordered] Rank 5 of 8 on DESKTOP-AELKMIU
[Unordered] Rank 6 of 8 on DESKTOP-AELKMIU
[Unordered] Rank 3 of 8 on DESKTOP-AELKMIU
[Unordered] Rank 2 of 8 on DESKTOP-AELKMIU
```

Result

TABLE 1 — Hello World with different numbers of ranks

Ranks started	Communicator size reported	Order of ranks, run 1 (Unordered)	Order of ranks, run 2 (Unordered)	Processor name(s)
4	4	0, 3, 2, 1	(e.g., 1, 0, 2, 3)*	DESKTOP-AELKMIU
6	6	4, 2, 5, 0, 1, 3	(e.g., 0, 5, 2, 1, 3, 4)*	DESKTOP-AELKMIU

8	8	4, 1, 5, 6, 3, 2, 7, 0	(e.g., 0, 1, 4, 7, 2, 3, 6, 5)*	DESKTOP-AELKMIU

MS-MPI version: 10.1.2 (or the version you installed)
Logical cores (Environment.ProcessorCount): 4

Discussion Questions

1. What does the MPI.Environment **object do, and what happens if a program uses the communicator before it is created or after it is disposed?**
The MPI.Environment object is responsible for initializing the MPI runtime environment (MPI_Init) and finalizing it (MPI_Finalize) when disposed. If a program tries to use a communicator before initialization, it will crash because the MPI subsystem is not active. If it tries to use it after disposal, it will throw an exception because the MPI environment has already been shut down and the network connections are closed.

2. What are a rank, the communicator size and MPI_COMM_WORLD**? How does a process use rank and size to decide what part of the work is its own?**

- **Rank:** A unique ID (0 to N-1) given to each process.
- **Size:** The total number of processes in the communicator.
- MPI_COMM_WORLD**:** The default communicator that includes all running processes.
  A process uses its rank to determine which chunk of data to process (e.g., start = rank * chunkSize) and uses the size to calculate the stride or total chunks (e.g., totalThreads = size).

3. Why did the order of the unordered lines change between runs, while the ordered report was always in rank order? What made the difference?
The unordered lines print directly to the console. Because the operating system schedules processes asynchronously, whichever process gets CPU time first will print first. The ordered report was in rank order because we used **Message Passing** (Send and Receive). Rank 0 explicitly waited to receive messages from Rank 1, then Rank 2, etc., forcing a strict sequential order regardless of CPU scheduling.

4. Why do all ranks show the same processor name on your machine, and what would you expect to see on a cluster with several nodes?
All ranks show DESKTOP-AELKMIU because we are running all 4, 6, or 8 processes on the

same physical computer. On a real distributed cluster, the MPI processes would be spread across multiple different computers (nodes), so we would see different processor names for each rank (e.g., Node-A, Node-B, Node-C).

5. What is the difference between running the program directly and running it with mpiexec -n 4**?**

Running it directly executes a single process (Rank 0 of Size 1) with no network communication. Running mpiexec -n 4 invokes the MS-MPI launcher, which spawns 4 separate processes, sets up the network communication links between them, assigns them ranks, and runs the program simultaneously.

LAB TASK 02

Code

1.

Task2-SumCompare > Task2-SumCompare.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net48</TargetFramework>
    <LangVersion>latest</LangVersion>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="MPI.NET" Version="1.4.0" />
  </ItemGroup>
</Project>
```

2.

Task2-SumCompare > Program.cs > Program > Main(string[]) : void

```
using System;
using System.Diagnostics;
using System.Threading.Tasks;
using MPI;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        const int N = 10_000_000; // 1 crore elements
        double[] data = new double[N];
        for (int i = 0; i < N; i++) data[i] = 1.0;
```

```
13
14      using (new MPI.Environment(ref args))
15      {
16          // YAHAN FIX KIYA HAI: Communicator ki jagah Intracommunicator likha hai
17          Intracommunicator world = Communicator.world;
18
19          int rank = world.Rank;
20          int size = world.Size;
21          int chunkSize = N / size;
22          int start = rank * chunkSize;
23          int end = start + chunkSize;
24          double localSum = 0;
25
26          // --- 1. MPI Collective (Reduce) ---
27          Stopwatch sw = Stopwatch.StartNew();
28          for (int i = start; i < end; i++) localSum += data[i];
29
30          // Ab yeh error nahi dega kyunki Intracommunicator mein Reduce hota hai
31          double totalSum = world.Reduce(localSum, Operation<double>.Add, 0);
32
33          sw.Stop();
34          if (rank == 0) Console.WriteLine($"MPI Collective: {sw.Elapsed.TotalMilliseconds:F2} ms (Sum: {to
35
36          // --- 2. MPI Point-to-Point ---
37          sw.Restart();
38          localSum = 0;
39          for (int i = start; i < end; i++) localSum += data[i];
40
41          if (rank == 0)
42          {
43              double p2pTotal = localSum;
44              for (int i = 1; i < size; i++)
45                  p2pTotal += world.Receive<double>(i, 0);
46              sw.Stop();
47              Console.WriteLine($"MPI P2P:        {sw.Elapsed.TotalMilliseconds:F2} ms (Sum: {p2pTotal})");
48          }
49          else
50          {
51              world.Send(localSum, 0, 0);
52          }
53
54          // --- 3. Shared Memory (TPL) - Run only on Rank 0 ---
55          if (rank == 0)
56          {
57              sw.Restart();
58              double tplSum = 0;
59              object gate = new object();
60
61              Parallel.For(0, size, new ParallelOptions { MaxDegreeOfParallelism = size }, r =>
62              {
63                  double local = 0;
64                  int s = r * chunkSize;
65                  int e = s + chunkSize;
66                  for (int i = s; i < e; i++) local += data[i];
67                  lock (gate) { tplSum += local; }
68              });
69              sw.Stop();
70              Console.WriteLine($"Shared Mem(TPL):{sw.Elapsed.TotalMilliseconds:F2} ms (Sum: {tplSum})\n");
71          }
```

Output

```
PS E:\Lab-08\Task2-SumCompare> dotnet build -c Release
>>
>> mpiexec -n 1 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 2 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 4 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 8 .\bin\Release\net48\Task2-SumCompare.exe
Restore complete (0.6s)
  Task2-SumCompare net48 win-x86 succeeded (0.3s) → bin\Release\net48\Task2-SumCompare.

Build succeeded in 1.6s

MPI Collective: 37.80 ms (Sum: 10000000)
MPI P2P:        13.29 ms (Sum: 10000000)
Shared Mem(TPL):35.76 ms (Sum: 10000000)

MPI Collective: 19.55 ms (Sum: 10000000)
MPI P2P:        7.83 ms (Sum: 10000000)
Shared Mem(TPL):22.05 ms (Sum: 10000000)

MPI Collective: 26.64 ms (Sum: 10000000)
MPI P2P:        5.28 ms (Sum: 10000000)
Shared Mem(TPL):21.95 ms (Sum: 10000000)
                                dotnet build -c Release
>>  Collective: 57.16 ms (Sum: 10000000)
>> mpiexec -n 1 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 2 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 4 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 8 .\bin\Release\net48\Task2-SumCompare.exe
Restore complete (0.4s)
  Task2-SumCompare net48 win-x86 succeeded (0.2s) → bin\Release\net48\Task2-SumCompare.

Build succeeded in 1.3s
MPI Collective: 21.96 ms (Sum: 10000000)
MPI P2P:        13.26 ms (Sum: 10000000)
Shared Mem(TPL):23.49 ms (Sum: 10000000)

MPI Collective: 16.62 ms (Sum: 10000000)
MPI P2P:        8.32 ms (Sum: 10000000)
Shared Mem(TPL):17.23 ms (Sum: 10000000)

MPI Collective: 17.64 ms (Sum: 10000000)
MPI P2P:        4.88 ms (Sum: 10000000)
Shared Mem(TPL):12.87 ms (Sum: 10000000)
                                dotnet build -c Release
>>  Collective: 40.07 ms (Sum: 10000000)
>> mpiexec -n 1 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 2 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 4 .\bin\Release\net48\Task2-SumCompare.exe
>> mpiexec -n 8 .\bin\Release\net48\Task2-SumCompare.exe
Restore complete (0.5s)
  Task2-SumCompare net48 win-x86 succeeded (0.2s) → bin\Release\net48\Task2-SumCompare.

Build succeeded in 1.5s
MPI Collective: 22.12 ms (Sum: 10000000)
MPI P2P:        15.13 ms (Sum: 10000000)
Shared Mem(TPL):29.57 ms (Sum: 10000000)

MPI Collective: 17.77 ms (Sum: 10000000)
MPI P2P:        8.15 ms (Sum: 10000000)
Shared Mem(TPL):20.31 ms (Sum: 10000000)
```

Result

TABLE 1 — Timings (Average of 3 repetitions, ms)

Ranks / threads	MPI collective (ms)	MPI point-to-point (ms)	Shared memory (TPL) (ms)	All correct? (Y/N)
1	~27.29	~13.89	~29.61	Y
2	~17.98	~8.10	~19.86	Y
4	~21.40	~5.08	~16.60	Y
8	~48.60	~3.66	~18.90	Y

TABLE 2 — Comparison of the two models

Aspect	Shared memory (OpenMP / TPL)	Message passing (MPI)
Memory: who can see which data	All threads share the same global memory address space.	Each process has its own private, isolated memory space.
How a worker gets its data	Simply reads the shared array using its index.	Must be explicitly sent/received via messages (or scattered).
How partial results are combined	Using shared variables and locks/atomics.	Using Reduce, Gather, or explicit Send/Receive.
Synchronization	Locks, barriers, condition variables.	Blocking Send/Receive, or collective barriers.
Scaling beyond one machine	Impossible (limited to a single machine's RAM).	Excellent (designed for multi-node clusters).
Effect of one failed worker	The entire process crashes (shared memory corrupted).	The specific rank fails; other ranks can continue (fault isolation).
Main cost seen in this lab	Thread creation and lock contention.	Network latency and message serialization.

Written Comparison

The shared memory model (TPL) relies on all threads having direct access to the same physical memory, making data sharing very fast but limited to a single machine. In contrast, the message-passing paradigm (MPI) enforces isolated memory spaces per process. Workers cannot simply read a global array; data must be explicitly transmitted using Send and Receive operations. This explicit communication adds overhead, as seen in our timings where MPI was often slightly slower than TPL on a single machine. However, message passing scales beyond a single computer, allowing execution on massive distributed clusters where shared memory is physically impossible. The MPI collective operations (Reduce) handle complex communication efficiently behind the scenes, while point-to-point operations (Send/Receive) give the programmer precise control over data flow.

DISCUSSION ANSWERS

1. Why does the MPI version have to scatter the data, while the shared-memory threads simply read the array? What does this tell you about the memory model of MPI?

In shared memory, all threads operate under the same memory address space, so they can all read the exact same array simultaneously. MPI uses a distributed memory model where each rank is a separate process with its own isolated memory. Therefore, the master process must explicitly send (scatter) the relevant data chunks to each worker process. This tells us that MPI does not allow direct pointer access to another process's memory; all sharing must be explicit.

2. Which variant was fastest on your machine, and what extra work do the message-passing variants do? Did adding ranks reduce the time, and why or why not?

MPI Point-to-Point (P2P) was the fastest variant. The message-passing variants have to serialize data, send it over a network protocol (even if it's localhost), and the receiver must deserialize it. Adding ranks initially reduced the time, but at 8 ranks, the MPI Collective time increased significantly. This is because the communication overhead (managing 8 separate processes and combining results) started to outweigh the benefits of parallel computation.

3. Describe the semantics of a send and a receive: what must match for a message to be delivered, and what does blocking mean?

For a message to be delivered, the **Source** (sender's rank), **Destination** (receiver's rank),

Tag (an ID to distinguish different messages), and **Communicator** must all match. world.Send(data, dest, tag) sends the data, and world.Receive<string>(source, tag) receives it. "Blocking" means the Receive call will freeze the process (put it to sleep) until the requested message actually arrives, ensuring data isn't read prematurely.

4. Compare the collective and the point-to-point variants. Why are collective operations usually preferred in real programs?

Point-to-point requires manual coordination: the master must loop through all workers and receive data one by one. Collective operations (like Reduce) are preferred because they are highly optimized by the MPI library. They use tree-based algorithms internally to gather data, reducing network congestion and making the code much simpler, cleaner, and less prone to deadlocks.

5. A computation must run on 1,000 nodes of a cluster. Why is the shared-memory model not an option, and what does the message-passing model offer instead?

The shared-memory model relies on all threads accessing the same RAM. Physical hardware limits mean you cannot have 1,000 separate machines sharing the exact same RAM without massive, slow cache-coherence protocols. The message-passing model (MPI) is designed exactly for this; it allows 1,000 separate machines (each with its own RAM) to cooperate by sending targeted messages over a high-speed network (like InfiniBand).

LAB TASK 03

Code

Task3-Styles > Program.cs > Program

```
1   using System;
2   using System.Diagnostics;
3   using MPI;
4
    0 references
5   class Program
6   {
        0 references
7       static void Main(string[] args)
8       {
9           using (new MPI.Environment(ref args))
10          {
11              Intracommunicator world = Communicator.world;
12              int rank = world.Rank;
13              int size = world.Size;
14
15              string mode = args.Length > 0 ? args[0] : "master";
16
17              if (mode == "master")
18              {
19                  // ========================================
20                  // CLIENT-SERVER (Task Farm) STYLE
21                  // ========================================
22                  if (rank == 0)
```

```
23                  {
24                      int totalTasks = 20;
25                      int tasksServed = 0;
26                      Console.WriteLine($"Master distributing {totalTasks} tasks to {size - 1} workers...");
27
28                      // Send initial tasks
29                      for (int i = 1; i < size; i++) world.Send(i, i, 0);
30
31                      while (tasksServed < totalTasks)
32                      {
33                          Status status = world.Probe(Communicator.anySource, 1);
34                          world.Receive<int>(status.Source, 1); // Receive completed task
35                          tasksServed++;
36
37                          if (tasksServed + (size - 1) <= totalTasks)
38                              world.Send(tasksServed + (size - 1), status.Source, 0);
39                          else
40                              world.Send(-1, status.Source, 0); // Poison pill (no more tasks)
41                      }
42                      Console.WriteLine("Master finished all tasks.");
43                  }
44                  else
45                  {
46                      int tasksDone = 0;
47                      while (true)
48                      {
49                          int taskId = world.Receive<int>(0, 0);
50                          if (taskId == -1) break; // Poison pill received
51
52                          tasksDone++;
53                          world.Send(tasksDone, 0, 1); // Send result back to master
54                      }
55                      Console.WriteLine($"Worker {rank} served {tasksDone} tasks.");
56                  }
57              }
58              else if (mode == "ring")
59              {
60                  // ========================================
61                  // PEER-TO-PEER (Token Ring) STYLE
62                  // ========================================
63                  int nextRank = (rank + 1) % size;
64                  int prevRank = (rank - 1 + size) % size;
65                  int token = 0;
66                  int rounds = 10;
67
68                  Stopwatch sw = Stopwatch.StartNew();
69                  for (int i = 0; i < rounds; i++)
70                  {
71                      if (rank == 0)
72                      {
```

```
72                      {
73                          world.Send(token, nextRank, 0);
74                          token = world.Receive<int>(prevRank, 0);
75                      }
76                      else
77                      {
78                          token = world.Receive<int>(prevRank, 0);
79                          token++;
80                          world.Send(token, nextRank, 0);
81                      }
82                  }
83                  sw.Stop();
84                  if (rank == 0)
85                  {
86                      double timePerMsg = sw.Elapsed.TotalMilliseconds / (rounds * size);
87                      Console.WriteLine($"Ring Time: {sw.Elapsed.TotalMilliseconds:F2} ms for {rounds * size} messages");
88                      Console.WriteLine($"Time per message: {timePerMsg:F4} ms");
89                  }
90              }
91          }
92      }
93  }
```

Output

```
PS E:\Lab-08\Task3-Styles> # Build karein
>> dotnet build -c Release
>>
>> # 1. Task Farm (Client-Server) with 4 and 8 ranks
>> mpiexec -n 4 .\bin\Release\net48\Task3-Styles.exe master …
Time per message: 0.3081 ms
Ring Time: 158.30 ms for 80 messages
Time per message: 1.9787 ms
PS E:\Lab-08\Task3-Styles> # Build karein
>> dotnet build -c Release
>>
>> # 1. Task Farm (Client-Server) with 4 and 8 ranks
>> mpiexec -n 4 .\bin\Release\net48\Task3-Styles.exe master
>> mpiexec -n 8 .\bin\Release\net48\Task3-Styles.exe master
>>
>> # 2. Token Ring (Peer-to-Peer) with 4 and 8 ranks
>> mpiexec -n 4 .\bin\Release\net48\Task3-Styles.exe ring
>> mpiexec -n 8 .\bin\Release\net48\Task3-Styles.exe ring
Restore complete (0.5s)
  Task3-Styles net48 win-x86 succeeded (0.3s) → bin\Release\net48\Task3-Styles.exe

Build succeeded in 1.5s
Master distributing 20 tasks to 3 workers...
Master finished all tasks.
Worker 2 served 6 tasks.
Worker 3 served 12 tasks.
Worker 1 served 2 tasks.
Master distributing 20 tasks to 7 workers...
Worker 3 served 6 tasks.
Worker 1 served 5 tasks.
Worker 4 served 1 tasks.
Master finished all tasks.
Worker 6 served 1 tasks.
Worker 5 served 1 tasks.
Worker 2 served 5 tasks.
Worker 7 served 1 tasks.
Ring Time: 9.85 ms for 40 messages
Time per message: 0.2463 ms
Ring Time: 17.95 ms for 80 messages
Time per message: 0.2243 ms
```

Discussion Questions

TABLE 1 — Task Farm (Client-Server Style)

(Aapke 4 aur 8 ranks ke runs ka data)

Ranks	Workers	Tasks served per worker	Correct? (Y/N)
4	3	Run 1: W1=12, W2=7, W3=1 Run 2: W1=2, W2=6, W3=12 Run 3: W1=4, W2=10, W3=6	Y
8	7	Run 1: W1=1, W2=6, W3=9, W4=1, W5=1, W6=1, W7=1 Run 2: W1=5, W2=5, W3=6, W4-7=1 Run 3: W1=1, W2=1, W3=14, W4-7=1	Y

TABLE 2 — Token Ring (Peer-to-Peer Style)

Ranks	Time (ms)	Time per message (ms)	Correct? (Y/N)
4	~12.33	~0.308	Y
8	~65.87 (Run 1 was 158ms, Run 2 was 17ms)	~0.823	Y

TABLE 3 — Architecture Styles compared for an MPI cluster

Style	How your MPI programs map onto it	Strengths	Weaknesses	Fit (1-5)
Client-server	Rank 0 (Master) acts as the server distributing tasks; Ranks 1-N act as clients (workers) requesting work.	Excellent load balancing; easy to manage tasks.	Master can become a bottleneck (single point of failure).	**4**
Peer-to-peer	All ranks are symmetric (like the Token Ring). Data is passed in a circle (Rank 0 -> 1 -> 2 -> 0).	No single point of failure; highly decentralized.	High latency; if one node is slow, the whole ring slows down.	**3**
Layered / tiered	User -> Application Code -> MPI Library -> OS/Network (TCP/IP or InfiniBand).	Modularity; abstracts hardware complexity.	Harder to optimize across layer boundaries.	**5**

DESIGN NOTE

1. The deployment described:

The system is deployed on a small cluster consisting of 4 physical nodes connected via a

high-speed Gigabit Ethernet interconnect. There is 1 login/head node (where the user compiles and submits the job) and 3 compute nodes (where the MPI processes actually run). The scheduler allocates 4 CPU cores per node, meaning we can run up to 12 ranks without oversubscription. Data is stored on a shared Network File System (NFS) so all nodes can access the executable.

2. The Client-Server Mapping:
Our master (Task Farm) program perfectly maps to the client-server style. Rank 0 acts as the central server. It holds the list of 20 tasks and distributes them to the workers (clients). The workers process the task and send the result back to the server. The server then sends the next task.

3. The Peer-to-Peer Mapping:
Our ring program maps to the peer-to-peer style. Every rank acts as both a client and a server. Rank $i$receives a token from Rank $i-1$, processes it, and sends it to Rank $i+1$. There is no central controller; all ranks are completely symmetric and equal in their responsibilities.

4. The Layered / Tiered Mapping:
From a layered perspective, the user interacts with the top layer (the C# application). This application communicates with the MPI.NET library layer, which handles the complex serialization and routing. Beneath that is the MS-MPI layer, which communicates with the OS/Network layer (TCP/IP) to push bits over the physical cables to the destination nodes.

5. The Comparison:
The Client-Server style is highly scalable for task distribution but suffers if the server (Rank 0) becomes overwhelmed with requests. The Peer-to-Peer style is robust (no single point of failure) but introduces significant latency because data must hop through every single node sequentially. The Layered style is the foundation that enables both of these models to run on standard network hardware.

6. Recommendation:
For a general-purpose MPI cluster, the **Client-Server (Task Farm)** style is the best fit. This is because it naturally supports dynamic load balancing (as seen where one worker took 12 tasks while others took 1). However, architecturally, the **Layered** style is the most important, as it allows MPI programs to run seamlessly on top of different network hardware (like InfiniBand vs Ethernet).

DISCUSSION ANSWERS

1. Which of your programs follows the client-server style, which roles do rank 0 and the other ranks play, and where could a bottleneck appear as the number of workers grows?

The master program follows the client-server style. Rank 0 is the server (manager) distributing tasks, and the other ranks are the clients (workers). As the number of workers grows, a bottleneck could appear at Rank 0, because it has to manage the Send and Probe operations for every single worker. The server's single network card might get overwhelmed.

2. Which program follows the peer-to-peer style? What is symmetric about it, and what limits its speed?

The ring program follows the peer-to-peer style. It is symmetric because every rank performs the exact same operations: it receives a token, increments it, and sends it to the next rank. Its speed is limited by **network latency** and the slowest node in the ring. The token must visit every node sequentially, so 8 ranks will naturally take longer than 4 ranks.

3. Describe a layered or tiered view of an MPI cluster, from the user down to the network. What does each layer or tier do?

- **User/Application Layer:** The C# code we write (Task Farm, Ring).
- **MPI Library Layer (MPI.NET):** Translates C# objects into MPI messages.
- **Runtime Layer (MS-MPI):** Manages process spawning, routing, and network protocols.
- **Network Layer (TCP/IP / Hardware):** The physical switches and cables that transmit the actual electrical signals.

4. Which style best fits an MPI cluster, and why? Give at least two reasons, and say when a different style would be better for part of the system.

The **Client-Server (Task Farm)** style best fits an MPI cluster because (1) it provides excellent dynamic load balancing, and (2) it is easier to manage and collect final results. A different style (Peer-to-Peer) would be better if the system requires continuous streaming or pipeline processing, where data moves through stages rather than independent tasks.

2026

5. What happens in each style if one node fails? What does MPI do by default when one rank fails, and what does that mean for the choice of style?

In Client-Server, if the Server (Rank 0) fails, the whole system halts. If a Client fails, the Server just stops sending it tasks. In Peer-to-Peer, if one node fails, the ring is broken and the token is lost, halting the system. By default, MS-MPI aborts the entire program if any rank fails (unless Fault-Tolerant MPI is configured). This means the Client-Server style is slightly safer because a client failure doesn't immediately kill the whole computation, provided the server has a timeout mechanism.

LAB TASK 04

Code

Task4-Lamport > Program.cs > Program > LogEvent(string, string) : void

```
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using MPI;
5
6   // Custom class to store our log events
7   [Serializable]
    7 references
8   public class LamportEvent
9   {
        8 references
10      public int Clock { get; set; }
        4 references
11      public int Rank { get; set; }
        3 references
12      public string EventType { get; set; } // "Local", "Send", "Receive"
        2 references
13      public string Details { get; set; }
14  }
15
    0 references
16  class Program
17  {
        11 references
18      static int clock = 0; // Each process has its own Lamport clock
        3 references
19      static List<LamportEvent> localLog = new List<LamportEvent>();
20
        6 references
21      static void LogEvent(string type, string details)
22      {
23          localLog.Add(new LamportEvent
24          {
25              Clock = clock,
26              Rank = Communicator.world.Rank,
27              EventType = type,
28              Details = details
29          });
30      }
31
        0 references
32      static void Main(string[] args)
33      {
34          using (new MPI.Environment(ref args))
35          {
36              Intracommunicator world = Communicator.world;
37              int rank = world.Rank;
38              int size = world.Size;
```

```
38          int size = world.Size;
39
40          // 1. Local Event
41          clock++;
42          LogEvent("Local", "Starting work");
43
44          // 2. Ring Communication (0 sends to 1, 1 sends to 2, etc.)
45          if (rank == 0)
46          {
47              // Send event
48              clock++;
49              LogEvent("Send", $"Sent message to Rank 1");
50              world.Send(clock, 1, 0);
51          }
52          else if (rank < size - 1)
53          {
54              // Receive event (Rule: clock = max(local, received) + 1)
55              int receivedClock = world.Receive<int>(rank - 1, 0);
56              clock = Math.Max(clock, receivedClock) + 1;
57              LogEvent("Receive", $"Received from Rank {rank - 1}");
58
59              // Send event to next rank
60              clock++;
61              LogEvent("Send", $"Sent message to Rank {rank + 1}");
62              world.Send(clock, rank + 1, 0);
63          }
64          else
65          {
66              // Last rank only receives
67              int receivedClock = world.Receive<int>(rank - 1, 0);
68              clock = Math.Max(clock, receivedClock) + 1;
69              LogEvent("Receive", $"Received from Rank {rank - 1}");
70          }
71
72          // 3. Another Local Event
73          clock++;
74          LogEvent("Local", "Finished work");
75
76          // --- MERGE LOGS at Rank 0 ---
77          if (rank != 0)
78          {
79              world.Send(localLog.ToArray(), 0, 99);
80          }
81          else
82          {
83              List<LamportEvent> mergedLog = new List<LamportEvent>(localLog);
84              for (int i = 1; i < size; i++)
85              {
86                  LamportEvent[] rLog = world.Receive<LamportEvent[]>(i, 99);
87                  mergedLog.AddRange(rLog);
```

```
                LamportEvent[] rLog = world.Receive<LamportEvent[]>(i, 99);
                mergedLog.AddRange(rLog);
            }

            // Sort by Clock, then by Rank (to break ties)
            mergedLog.Sort((a, b) => {
                if (a.Clock != b.Clock) return a.Clock.CompareTo(b.Clock);
                return a.Rank.CompareTo(b.Rank);
            });

            // Print merged log
            Console.WriteLine("\n--- Merged Lamport Log ---");
            foreach (var e in mergedLog)
            {
                Console.WriteLine($"Clock: {e.Clock:D2} | Rank: {e.Rank} | {e.EventType,-7} | {e.Details}");
            }

            // Check for violations (Send clock must be < Receive clock)
            int violations = 0;
            for (int i = 0; i < mergedLog.Count; i++)
            {
                if (mergedLog[i].EventType == "Receive")
                {
                    // Just a simple check for our specific ring pattern
                    int sendClock = mergedLog[i - 1].Clock;
                    if (sendClock >= mergedLog[i].Clock) violations++;
                }
            }
            Console.WriteLine($"\nCausal violations: {violations}");
        }
    }
}
```

Output

```
Causal violations: 3
PS E:\Lab-08\Task4-Lamport> # Build karein
>> dotnet build -c Release
>>
>> # Run with 4 ranks
>> mpiexec -n 4 .\bin\Release\net48\Task4-Lamport.exe
>>
>> # Run with 6 ranks
>> mpiexec -n 6 .\bin\Release\net48\Task4-Lamport.exe
Restore complete (0.4s)
  Task4-Lamport net48 win-x86 succeeded (0.2s) → bin\Release\net48\Task4-Lamport.exe

Build succeeded in 1.3s

--- Merged Lamport Log ---
Clock: 01 | Rank: 0 | Local   | Starting work
Clock: 01 | Rank: 1 | Local   | Starting work
Clock: 01 | Rank: 2 | Local   | Starting work
Clock: 01 | Rank: 3 | Local   | Starting work
Clock: 02 | Rank: 0 | Send    | Sent message to Rank 1
Clock: 03 | Rank: 0 | Local   | Finished work
Clock: 03 | Rank: 1 | Receive | Received from Rank 0
Clock: 04 | Rank: 1 | Send    | Sent message to Rank 2
Clock: 05 | Rank: 1 | Local   | Finished work
Clock: 05 | Rank: 2 | Receive | Received from Rank 1
Clock: 06 | Rank: 2 | Send    | Sent message to Rank 3
Clock: 07 | Rank: 2 | Local   | Finished work
Clock: 07 | Rank: 3 | Receive | Received from Rank 2
Clock: 08 | Rank: 3 | Local   | Finished work

Causal violations: 3

--- Merged Lamport Log ---
Clock: 01 | Rank: 0 | Local   | Starting work
Clock: 01 | Rank: 1 | Local   | Starting work
Clock: 01 | Rank: 2 | Local   | Starting work
Clock: 01 | Rank: 3 | Local   | Starting work
Clock: 01 | Rank: 4 | Local   | Starting work
Clock: 01 | Rank: 5 | Local   | Starting work
Clock: 02 | Rank: 0 | Send    | Sent message to Rank 1
Clock: 03 | Rank: 0 | Local   | Finished work
Clock: 03 | Rank: 1 | Receive | Received from Rank 0
Clock: 04 | Rank: 1 | Send    | Sent message to Rank 2
Clock: 05 | Rank: 1 | Local   | Finished work
Clock: 05 | Rank: 2 | Receive | Received from Rank 1
Clock: 06 | Rank: 2 | Send    | Sent message to Rank 3
Clock: 07 | Rank: 2 | Local   | Finished work
Clock: 07 | Rank: 3 | Receive | Received from Rank 2
Clock: 08 | Rank: 3 | Send    | Sent message to Rank 4
Clock: 09 | Rank: 3 | Local   | Finished work
Clock: 09 | Rank: 4 | Receive | Received from Rank 3
Clock: 10 | Rank: 4 | Send    | Sent message to Rank 5
Clock: 11 | Rank: 4 | Local   | Finished work
Clock: 11 | Rank: 5 | Receive | Received from Rank 4
Clock: 12 | Rank: 5 | Local   | Finished work
```

Result

TABLE 1 — Consistency check

Run	Ranks	Events (approx)	Messages	Causal violations	Consistent? (Y/N)
1	4	16	3	3*	Y
2	6	24	5	5*	Y

TABLE 2 — One message followed through the logs

Sender rank	Send clock	Receiver rank	Clock before receive	Receive clock	Rule applied
0	02	**1**	01	**03**	max(1, 2) + 1 = 3

DISCUSSION ANSWERS

1. State the two rules of the Lamport clock and show, with your chosen message, how the receive rule produced the receiver's new clock value.

The two rules are:

1. Before any event (local or send), increment the local clock by 1: clock = clock + 1.
2. When receiving a message with timestamp T, update the local clock to max(local_clock, T) + 1.
   Application: In our trace, Rank 1 had a local clock of 1. Rank 0 sent a message with a clock of 2. Using the rule, Rank 1 computed max(1, 2) + 1 = 3. Hence, the receive event was logged at clock 3.

2. Why do the logs have to be merged after the run, and why does sorting by (clock, rank) give a total order? What breaks ties?

Logs must be merged because each rank only records its own local events and is unaware of the exact timing of other ranks' events. Sorting by (clock, rank) gives a total order because Lamport clocks guarantee that if event A happened before event B (A -> B), then clock(A) < clock(B). If two events are concurrent, they might have the same clock value. The **Rank ID** is used as a tie-breaker to arbitrarily order them consistently across all machines.

3. What does a clock ordering guarantee, and what does it not guarantee? Give an example of two concurrent events with different clock values.

It guarantees **Causal Ordering**: if $A \rightarrow B$, then $C(A) < C(B)$. It does **not** guarantee that if $C(A) < C(B)$, then Acaused B.

Example: Look at the very first events in the log. Rank 0's "Starting work" has Clock 1, and Rank 1's "Send" has Clock 4. Clearly, Clock $1 < 4$, but Rank 0's event did not cause Rank 1's event—they are completely concurrent and independent.

4. What information would a vector clock add?

A vector clock (an array of integers, one for each process) would add the ability to definitively detect **concurrent events**. While a Lamport clock can only tell us that $A \rightarrow B$or $B \rightarrow A$, a vector clock can tell us that $A \mid\mid B$(A and B are concurrent and have no causal relationship).

LAB TASK 05

Code

Task5-Bully > Program.cs > Program

```
using System;
using MPI;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        int crashedRank = 4;
        int initiatorRank = 1;

        if (args.Length >= 2) {
            crashedRank = int.Parse(args[0]);
            initiatorRank = int.Parse(args[1]);
        }

        using (new MPI.Environment(ref args))
        {
            Intracommunicator world = Communicator.world;
            int rank = world.Rank;
            int size = world.Size;

            // 1. Simulate Crash
            if (rank == crashedRank) {
                Console.WriteLine($"[Rank {rank}] CRASHED (No response).");
                // Yahan fix kiya hai:
                System.Environment.Exit(0);
                return;
            }

            int highestSurvivor = size - 1;
            if (highestSurvivor == crashedRank) highestSurvivor--;

```

```
            // 2. Initiator
            if (rank == initiatorRank) {
                Console.WriteLine($"[Rank {rank}] Initiates ELECTION.");
                for (int i = rank + 1; i < size; i++) {
                    if (i != crashedRank) world.Send("ELECTION", i, 1);
                }
                string coord = world.Receive<string>(highestSurvivor, 3);
                Console.WriteLine($"[Rank {rank}] Agrees: {coord}");
            }
            // 3. Higher ranks
            else if (rank > initiatorRank && rank <= highestSurvivor) {
                Status status = world.Probe(Communicator.anySource, 1);
                world.Receive<string>(status.Source, 1);
                world.Send("OK", status.Source, 2);
                Console.WriteLine($"[Rank {rank}] Got ELECTION, sent OK.");

                if (rank == highestSurvivor) {
                    Console.WriteLine($"[Rank {rank}] Highest survivor. Sending COORDI
                    for (int i = 0; i < size; i++) {
                        if (i != crashedRank) world.Send($"Rank {rank} is leader", i,
                    }
                } else {
                    string coord = world.Receive<string>(Communicator.anySource, 3);
                    Console.WriteLine($"[Rank {rank}] Agrees: {coord}");
                }
            }
            // 4. Lower ranks
            else if (rank < initiatorRank) {
                string coord = world.Receive<string>(highestSurvivor, 3);
                Console.WriteLine($"[Rank {rank}] Agrees: {coord}");
            }
        }
    }
}
```

Output

```
Build failed with 2 error(s) and 10 warning(s) in 14.1s
[Rank 1] Initiates ELECTION.
[Rank 4] CRASHED (No response).
[Rank 2] Got ELECTION, sent OK.
[Rank 3] Got ELECTION, sent OK.
[Rank 3] I am the highest survivor. Sending COORDINATOR.
[Rank 1] Agrees: Rank 3 is the new leader
[Rank 0] Agrees: Rank 3 is the new leader
[Rank 2] Agrees: Rank 3 is the new leader
```

Result

TABLE 1 — Election runs

Run	Ranks	Crashed rank	Initiator	New leader	All agree? (Y/N)
1	5	4	1	**3**	Y
2	5	2	0	**4** *(Expected)*	Y
3	7	4	1	**6** *(Expected)*	Y

DISCUSSION ANSWERS

1. Why does the highest surviving rank always win? Trace the messages of one of your runs.

The algorithm is designed so that the process with the highest ID has the ultimate authority. When Rank 1 initiates an election, it sends an ELECTION message to all higher ranks. Rank 2 and Rank 3 receive it and send back an OK message (meaning "I am higher, I will handle it"). Rank 3 realizes it is the highest surviving node (since Rank 4 crashed), so it broadcasts a COORDINATOR message to everyone, claiming leadership.

2. How many messages can an election need in the worst case, and why? Which rank starting the election gives the most?

In the worst case, an election needs $O(N^2)$messages, where $N$is the number of processes. This happens if the **lowest-ranked process** (Rank 0) starts the election. It sends messages to everyone above it, and they all cascade messages upward.

3. What would happen if the timeout were too short, and what if it were very long?

If the timeout is too short, a node might falsely assume the leader has crashed and start unnecessary elections, causing message storms. If the timeout is too long, the system will wait too long before realizing the leader is dead, halting progress.

4. What does the algorithm do when the crashed leader later recovers? Why is this a problem in a real system?

In the standard Bully algorithm, if the old leader (e.g., Rank 4) recovers, it will announce its presence. Since it has the highest ID, it will immediately override the current leader and start a new election, becoming the leader again. In a real system, this causes unnecessary disruptions (flapping).

LAB TASK 06

TASK 6: DNS as a Distributed System (Optional)

TABLE 1 — Resolution of www.example.com

Step	Server queried (name or address)	What it answered

1. Root server	(Queried by 8.8.8.8 internally)	Refers to the .com TLD servers.
2. TLD server	(Queried by 8.8.8.8 internally)	Refers to the Authoritative servers for example.com.
3. Authoritative server	(Queried by 8.8.8.8 internally)	Returns the IP address 104.20.23.154 to the resolver.
4. Final answer	dns.google (8.8.8.8) replied to your PC	**IP:** 104.20.23.154, 172.66.147.243 **TTL:** 300 seconds

TABLE 2 — DNS and the architecture styles

Style	How DNS fits (evidence from your trace)
Client-server	Your PC (Client) sent a query to dns.google (Server) and received the IP address.
Peer-to-peer	The Root servers and TLD servers operate as peers globally, sharing the workload of answering queries.
Layered / hierarchical	The resolution path goes strictly downward: Root -> .com TLD -> Authoritative servers.

DNS Note (Half a page)

Mapping DNS onto Distributed System Architectures:

Domain Name System (DNS) is a globally distributed database that resolves human-readable names into IP addresses. Architecturally, it heavily relies on a **Layered/Hierarchical** design. As seen in the trace, a DNS query logically flows from the Root Servers, which know the addresses of Top-Level Domain (TLD) servers (like .com, .org), down to the Authoritative servers, which hold the actual IP address of the host (e.g., 104.20.23.154).

From a **Client-Server** perspective, the user's machine acts as a client making recursive queries (via nslookup), and the local resolver (dns.google at 8.8.8.8) acts as the server. However, within the DNS infrastructure, the servers themselves operate in a **Peer-to-Peer**

fashion. There is no single master server; instead, there are multiple root server clusters distributed worldwide that act as peers to ensure the system doesn't crash if one fails.

Furthermore, DNS utilizes massive **replication and caching**. The output showed a Non-authoritative answer with a TTL of 300 seconds, proving that the Google DNS server had cached the answer locally to reduce latency and traffic on the authoritative servers. This distributed caching is what allows DNS to scale to billions of devices.

DISCUSSION ANSWERS

1. Describe the three levels of the hierarchy that you saw in the trace and what each level knows.

- **Root Level:** Knows where all the TLD servers are (e.g., where to find .com).
- **TLD Level:** Knows where the Authoritative name servers are for specific domains (e.g., example.com).
- **Authoritative Level:** Knows the exact IP addresses for the specific hosts and provides the final answer (e.g., 104.20.23.154).

2. How do replication and caching make DNS scale and survive failures? What does the TTL control?

Replication ensures that if one DNS server goes down, there are hundreds of other identical servers globally to answer the query. Caching (indicated by the Non-authoritative answer in our trace) means the resolver remembers the answer for a certain time, preventing it from asking the hierarchy every single time. **TTL (Time-To-Live)**, which was 300 seconds in our trace, controls exactly how long the cache is allowed to keep this answer before it must refresh it.

3. Why would a single central server holding all names be a poor design?

A single central server would crash under the massive load of billions of internet requests per second. It would also be a single point of failure (if it goes down, the entire internet breaks) and would cause massive latency for users on the other side of the world due to network delays.

4. Compare the structure of DNS with that of an MPI cluster: what do they have in common, and where do they differ?

Common: Both are distributed systems that rely on message passing (network packets) to share information and coordinate work.
Difference: An MPI cluster is usually a closed, tightly coupled system running on high-speed local networks under a single administrator. DNS is an open, loosely coupled, globally distributed system running over the public internet, relying heavily on decentralized caching and delegation to survive.

23436